Criiq — Viva / Project Defense Q&A

Project-specific answers, traced to the actual codebase and validation numbers

Sushma Shrestha

13 August 2026

Contents

How to use this document **4**

1. Project Overview, Motivation & Scope **5**
Q1. CRIIQ bhaneko ke ho ra cricket selection ma yesle kun specific problem solve garcha? 5
Q2. Tapai le women's cricket lai primary focus kina garnu bhayo? 5
The monetary reality 5
The incremental value 5
Real-life goals and problems 6
Q2-follow-up. Yesto samasya cinha pachi, hamro project le tyo kasari solve garyo, ra hamro contribution ke ho? 6
Q3. Existing cricket analytics systems ko comparison ma tapai ko project lai novel banaune kura ke ho? 8
Q4. Yo system develop garnu ko main motivation ke thiyo? 8
Q5. Tapai ko system traditional selector ko decision-making process bhanda kasari different cha? . 8
Q6. Tapai ko system ko target users ko hun ra kina? 9
Q7. Tapai ko project ko main objectives ke-ke thiye? 9
Q8. Tapai ko system le available cricket data ra actual player selection practice bich ko gap kasari address garcha? 9
Q63. CRIIQ ko purpose selectors lai replace garnu ho ki decision-support provide garnu ho? 10

2. Data Pipeline & Sources **11**
Q9. Tapai ko data sources ke-ke hun ra harek source kina choose garnu bhayo? 11
Q10. Multiple data sources bata aayeko data ko quality kasari ensure garnu bhayo? 11
Q11. Different sources ma player identity resolution kasari handle garnu huncha? 12
Q12. Tapai ko data pipeline ko overall structure kasto cha? 12
Q13. Data pipeline lai idempotent ra safely re-run garna milne kasari ensure garnu huncha? 13
Q14. Missing data, especially less-covered players ko case ma, kasari handle garnu huncha? 13
Q15. Recent Form Index bhaneko ke ho ra yeslai kasari calculate garnu huncha? 13

3. Feature Engineering & Data Leakage **14**
Q16. Feature engineering garda data leakage kasari prevent garnu huncha? 14

4. Win-Probability-Added (WPA) — the Core Modeling Idea **15**
Q17. WPA bhaneko ke ho ra raw batting/bowling statistics ko satta WPA kina use garnu bhayo? . . 15
Q18. WPA ko mathematical formula formulate garnus ra harek term explain garnus. 15
Q19. Per-ball win probability kasari calculate garnu huncha? 15
Q20. Batting ra bowling WPA lai separate targets ko rupma kina train garnu bhayo? 16
Q21. Tapai ko WPA regression model le kun-kun features use garcha? 16
Q22. HistGradientBoostingRegressor nai kina choose garnu bhayo? 17

Q23. Different machine learning models lai kasari compare garnu bhayo? . . . 17
Q24. Temporal holdout bhaneko ke ho ra random train-test split ko satta yeslai kina use garnu bhayo? . . . 18
Q25. Tapai ko R^2 values 0.015 bhanda tala chan. Yeslai kasari justify garnu huncha? . . . 18
Q26. Pearson correlation R^2 bhanda tapai ko use case ma kina important cha? . . . 18
Q27. Brier score bhaneko ke ho ra yeslai kina use garnu huncha? . . . 18
Q28. Sign agreement accuracy le ke measure garcha ra yo kina important cha? . . . 19
Q29. A few matches bhayeka players ko lagi shrinkage ko role ke ho? . . . 19
Q30. Predicted vs. actual WPA plot lai kasari interpret garnu huncha? . . . 20
Q31. Residual plots le model ko bias ko barema ke information dincha? . . . 20

5. The Composite Recommendation Score **21**
Q32. Tapai ko composite recommendation score ko seven components ke-ke hun? . . . 21
Q33. Model output matra use garnu ko satta composite score kina use garnu bhayo? . . . 21
Q34. Composite score ko weights kasari determine garnu bhayo? . . . 21
Q35. Matchup component bhaneko ke ho ra yeslai kasari calculate garnu huncha? . . . 22
Q36. Need component bhaneko ke ho ra yesle team balance kasari improve garcha? . . . 22
Q37. Risk penalties ke-ke hun ra kahile apply hunchan? . . . 22
Q38. Phase component bhaneko ke ho ra player ko best phase kasari identify garnu huncha? . . . 23
Q39. WPA model component lai 0–45 score ma kasari normalize garnu huncha? . . . 23
Q40. Composite score le raw WPA model bhanda better selection garcha bhanera kasari test garnu bhayo? . . . 23

6. Playing XI Selection — Mixed-Integer Linear Programming (MILP) **24**
Q41. XI selection problem lai mathematically kasari formulate garnu huncha? . . . 24
Q42. Top 11 scores matra select garnu ko satta MILP kina use garnu bhayo? . . . 24
Q43. Legal playing XI ko lagi kun-kun constraints enforce garnu huncha? . . . 24
Q44. Insufficient qualifying players, Bhayeko pool lai kasari handle garnu huncha? . . . 24
Q45. MILP solver fail bhayo bhane ke huncha? . . . 25
Q46. Greedy repair fallback bhaneko ke ho ra kahile use huncha? . . . 25
Q47. MILP solver ko correctness kasari test garnu huncha? . . . 25
Q48. All-rounders jasto multiple roles fulfill garna sakne players lai kasari handle garnu huncha? . . . 25

7. Genetic Algorithms — Bowling Rotation & Batting Order **27**
Q49. Genetic Algorithm le system ma ke optimize garcha? . . . 27
Q50. Bowling rotation GA le kaam garyo tara batting-order GA le kina garena? . . . 27
Q51. Genetic Algorithm ko convergence kasari evaluate garnu huncha? . . . 27
Q52. Bowling rotation problem ko solution kasari encode garnu huncha? . . . 28
Q53. Batting-order GA ko performance limitation kina bhayo? . . . 28
Q54. Batting-order GA lai improve garna ke garnu parcha? . . . 28

8. Explainability — SHAP and GA Explanations **29**
Q55. SHAP explainability tapai ko system ma kasari work garcha? . . . 29
Q56. SHAP lai aru explainability methods bhanda kina choose garnu bhayo? . . . 29
Q57. SHAP explanations correct ra non-misleading chan bhanera kasari validate garnu huncha? . . . 29
Q58. Selectors ko lagi explainability kina important cha? . . . 30

9. Technology Stack **31**
Q59. Python, FastAPI, React, SQLite, Parquet, and Joblib—jasto technology haru kina choose garnu bhayo? . . . 31

10. Limitations, Deployment & Future Work **32**
Q60. Tapai ko system ko main limitations ke-ke hun? . . . 32
Q61. Future work ma tapai le ke-ke improve garna plan garnu bhayeko cha? . . . 32
Q62. Actual cricket board ma yo system kasari deploy garnu huncha? . . . 33

11. Reflective Questions (for you to answer, grounded in real facts) **34**
Q64. Project ma tapai ko personal contribution ke thiyo? 34
Q65. Tapai le project bata sabai bhanda important learning ke gain garnu bhayo? 34

Appendix — Source Map **35**

How to use this document

Every answer below is traced to an actual file, formula, or measured number already present in the repository — mainly `docs/criiq-technical-documentation.md`, `model-training/ALGORITHMS.md`, `model-training/README.md`, `model-training/VALIDATION_REPORT.md`, `data-collection/README.md`, and `WORKING.md`. Where a question's premise doesn't match what the code actually does, that is said plainly rather than invented — this mirrors the project's own "honest reporting" convention (every one of those five documents follows the same rule: report what was measured, not what would sound better).

The last section (§11, "Reflective questions") covers two questions only you can answer truthfully — your personal contribution and your biggest learning. Fabricating those would defeat the purpose of a viva, so that section gives you the real, verifiable facts to build your own answer from, not a ready-made script.

1. Project Overview, Motivation & Scope

Q1. CRIIQ bhaneko ke ho ra cricket selection ma yesle kun specific problem solve garcha?

Criiq is a cricket analytics and **player-selection recommendation system**. Given a fixture context — "*Team A is playing Team B, optionally at venue V*" — it scores every eligible player in Team A's real, point-in-time candidate pool and proposes a constrained, balanced Playing XI, with a plain-language reason attached to every pick (`docs/criiq-technical-documentation.md`, "What Criiq is").

The specific problem it targets: real selection decisions are made from **scattered, partial, and often stale information** — a selector has to mentally combine career stats, recent form, head-to-head history, venue record, and current fitness/status, none of which live in one place, and much of which (e.g. "is she still actually in form domestically, even though she's been quiet internationally") is easy to miss entirely. Criiq's concrete deliverable is two things that were directly validated against real, unseen 2026 matches (`VALIDATION_REPORT.md` §11): the recommended XI overlapped the real historical starting XI by **55.8%** of players (vs. 37.2% for a random-from-pool baseline), and the match simulator called the real winner **66.7%** of the time on real held-out matches — both clearly better than chance, neither claiming to replace human judgment outright.

Q2. Tapai le women's cricket lai primary focus kina garnu bhayo?

Three real things line up here — what the format is now worth, what a data-driven selection process is actually worth wherever it's been tried, and a real, current problem in exactly this sport — and together they make "women's cricket first" a harder-nosed decision than it might first sound, not a purely symbolic one.

The monetary reality

Women's cricket stopped being a financial footnote very recently, and the numbers are large enough that a selection decision inside this format now has real money riding on it, not just pride:

- The **2023 Women's Premier League** auction raised **₹4,669.99 crore (~US $572 million)** for five franchises on day one, backed by a **₹951 crore (~US $116 million)** media-rights deal — and viewership **more than doubled** the very next season.
- The **ICC moved to fully equal prize money** for men's and women's T20 World Cups starting in 2024 — seven years ahead of its own original 2030 target. The 2024 Women's T20 World Cup prize pool was **US $7.96 million**, a **225% increase** on the previous edition; the winning team alone took **US $2.34 million**, more than double the previous champions' payout.
- Sponsorship and ad rates for the **2025 ICC Women's Cricket World Cup** surged **up to 50%** over 2022, with advertising revenue expected to rise **70%**, driven directly by the WPL's success.
- Globally, women's elite sports revenue hit **US $1.88 billion in 2024** and was projected to cross **US $2.35 billion in 2025** — commercial income alone passed **US $1 billion** for the first time in 2024.

None of this was true even five years ago. A system that treats women's T20I as an afterthought would be building for a version of this sport that no longer exists.

The incremental value

The value of data-driven selection over gut-feel selection isn't a hypothetical in sport generally — it's one of the most measured effects in modern sports analytics. Baseball's "Moneyball" era is the canonical case: a team spending a fraction of its rivals' salaries became competitive by picking players on the metrics that actually predicted winning rather than the ones that merely looked impressive. More broadly, a 2022 study (*IJSS*) found teams that actually use data **win championships 63% more often** than those that don't, and today **97% of professional teams** employ dedicated data staff — data-driven selection isn't an edge case anymore, it's the baseline expectation almost everywhere except here.

That's the *incremental* argument specifically: women's cricket's analytics infrastructure is still comparatively young — this project's own data-collection layer runs straight into real, current gaps (ball-tracking, shot-coding, and pitch-map data are honestly logged as **"essentially non-existent for women's T20I / associate cricket,"** `data-collection/README.md`, "What is NOT obtainable") that men's cricket closed years ago. That means the *marginal* value of even a modest, honestly-validated improvement in selection quality is larger here, right now, than it would be bolting the same idea onto a format that's already saturated with scouting tools — there is simply more low-hanging fruit left unpicked.

Real-life goals and problems

The problem this is actually aimed at is not hypothetical either — informal, subjective selection is a live, current, publicly-documented source of controversy in exactly this sport:

- **Mithali Raj**, India's most-capped women's player, was dropped for the 2018 Women's World T20 semi-final and publicly alleged discriminatory treatment from the team's own coach — one of the most visible selection controversies in the format's history.
- Former India selector **Krishnamachari Srikkanth** publicly accused the national selection committee of "too much favoritism" as recently as 2024; an all-rounder's own father alleged bias against his son in 2025; Zimbabwe's selectors faced a near-identical "bias" accusation the same year, in a leaked recording.

These aren't edge cases — they're recurring, and they share the same root cause: a selection process with no transparent, checkable, number-backed reasoning behind it invites exactly this kind of dispute, because there's nothing concrete to point to either way. That's the real-life goal this project is built around: not to remove a human selector's judgment, but to give a selection decision a transparent, auditable, number-backed foundation it can be defended (or corrected) against — a specific, scoped answer to a problem that currently plays out as public accusations of favoritism, in a format that (per the figures above) now has genuine money and careers riding on getting it right.

(All three sections above are real and independently citable — the one sentence a jury will actually remember is the personal one only you can write: if there's a specific player, match, or moment that made this feel real to you rather than statistical, say that here instead of leaving this as a placeholder.)

External sources (not from the codebase, cited separately so they can be checked independently):

- WPL 2023 franchise auction and media-rights valuation — Businesswire, BCCI WPL auction release, WPL Net Worth 2026
- WPL viewership growth — The Streaming Lab, "Evaluating Women's Sports Streaming Across India"
- ICC equal prize money, 2024 Women's T20 World Cup pool — ICC, "ICC enters era of equal prize money for men and women", BBC Sport, "ICC announces record prize money for Women's T20 World Cup"
- 2025 Women's Cricket World Cup sponsorship surge — Exchange4media, "ICC Women's Cricket World Cup 2025: Sponsorship rates surge up to 50%"
- Global women's sports revenue — Deloitte, "Women's elite sports revenues to surpass US $2.35 billion in 2025"
- Ball-tracking coverage timeline for ICC women's tournaments — Emergency Cricket, "Women's cricket – fastest observed bowling speeds"
- Data-driven selection's measured effect on winning — IJSS study cited in discussion of sports analytics adoption; see also general "Moneyball" sabermetrics literature
- Selection-bias controversies — Quora, "What are the biggest criticisms of the Indian women's cricket team at the ICC World Cup?", AllAfrica, "Zimbabwe Cricket Selectors Accused of 'Bias'…"

Q2-follow-up. Yesto samasya cinha pachi, hamro project le tyo kasari solve garyo, ra hamro contribution ke ho?

Worth being precise about scope before claiming anything: this project doesn't move the money above, and it doesn't end selection controversies in cricket — no academic system could honestly claim either. What it

does do is build a concrete, **working, measured mechanism aimed at each of the three problems above**, and then check, on real data, whether that mechanism actually holds up rather than just asserting that it should. That distinction — a checked contribution, not a claimed one — is the honest version of "how it solved this."

Against the infrastructure/incremental-value gap: the project didn't just note the gap, it closed specific pieces of it, for women's cricket specifically, and measured the closing:

- **The "quiet internationally, in-form domestically" blind spot is closed, not just described.** Seven women's domestic T20 leagues (WBBL, WPL, The Hundred Women's, Women's T20 Blast/Challenge, WCSL, T20 Blaze) are ingested and turned into a real `player_domestic_form` signal, kept in its own table so domestic-league scale never corrupts the international career line. Concrete, real, non-cherry-picked proof it works as intended: `GA Elwiss`, `A Wellington`, and `L Winfield` — all absent from international cricket for 3+ years, all recently active in WBBL/The Hundred — are now correctly resolved to their real country (not dropped as "inactive") and appear in their country's eligible pool, exactly the player this feature exists to stop a selector from losing track of.
- **A specific women's-cricket data-coverage gap was measurably closed.** Bowling style (needed for vs-pace/vs-spin matchup scoring) was resolvable for only **127 of 1,739 real female T20 bowlers (7.3%)** before this project's Wikipedia/Wikidata enrichment pass; running it at scale raised that to **568 (32.7%)** — a four-fold real increase in exactly the kind of coverage the "incremental value" argument says is still low-hanging fruit here.
- **A bug that would have mis-scored women specifically was caught, not shipped.** Because the archetype classifier (power hitter / finisher / anchor) was built and checked against women's data as a first-class case rather than an afterthought, an absolute strike-rate/six-rate threshold that would have tagged almost every man a "power hitter" and almost no woman was caught before shipping, and replaced with a gender-relative percentile bar.

Against the selection-bias/controversy problem — the most direct answer: the entire recommendation is built to be an auditable number, not a private judgment call, which is the structural opposite of what fuelled the disputes cited above (Mithali Raj's 2018 exclusion, the 2024/2025 favoritism allegations):

- **Every point in a player's score traces to a named, capped, inspectable component** (`recommend/score.py`'s seven-part composite, Q32) with its raw inputs attached — `explain.py`'s hard rule is that it "never states anything that isn't in the computed numbers." A dropped player's rating is a specific, checkable number with reasons, not a closed-door panel decision.
- **SHAP gives an exact, mathematically-verified attribution** for the model component specifically (`base_value + Σcontributions = the actual prediction`, checked by an automated test that would fail immediately if the decomposition ever drifted from the real number — Q57) — "why was she rated this way" has a real, reproducible answer, not a plausible-sounding story.
- **`analyze_xi` is the direct, purpose-built answer to a Mithali-Raj-shaped dispute.** It takes a selector's *actual* chosen XI, scores it against the exact same constraints and the same real candidate pool, and reports the precise score gap against the provably-optimal MILP pick plus exactly which players differ and why — turning "was leaving her out justified" from a public argument with no agreed facts into one specific, reproducible, challengeable number. It doesn't force a selector's hand; it gives both the selector and their critics the same concrete reference point to argue from, which is what none of the disputes cited above had.
- **This was checked against real history, not just asserted.** `VALIDATION_REPORT.md` §11 ran the full recommendation-plus-MILP pipeline against 149 genuinely unseen 2026 matches (including the entire ICC Women's T20 World Cup) the models had never trained on: the recommended XI overlapped the real historical starting XI by **55.8%** on average, against a **37.2%** random-from-pool baseline — a real, measured margin, not a claim taken on faith. Two full end-to-end simulations, including the actual **World Cup Final**, both correctly called the real winner.

The honest limit of this contribution: this is a validated prototype, not a system any cricket board has adopted (Q62's own answer is explicit that the current deployment is a self-contained demo bundle, not a production rollout). It doesn't remove a human selector's judgment or a board's final call, and it inherits real,

stated weaknesses of its own — the underlying WPA signal is genuinely noisy (correlation 0.07–0.12, Q25), and coverage (bowling style, wicket-keeper flags) is still partial. The honest claim is narrower than "this fixes cricket selection": it is a working, checked demonstration that a transparent, number-backed alternative to gut-feel selection is buildable for women's cricket specifically, that it measurably beats a naive baseline on real matches, and that it directly targets the exact failure mode (opaque, undefendable decisions) behind the real disputes cited above — not a claim that the underlying problem is now solved industry-wide.

Q3. Existing cricket analytics systems ko comparison ma tapai ko project lai novel banaune kura ke ho?

The codebase doesn't compare itself to any named commercial product (CricViz, ESPNcricinfo Smart Stats, etc. are never referenced in code or docs), so the honest answer is about the concrete, checkable design choices rather than a competitive claim:

- **Win-Probability-Added (WPA) as the actual training target**, not raw contribution-share — validated by a real, documented failure case (§4 below) and checked against a learning-to-rank alternative on real data (§4, `evaluate/rank_backtest.py`) before being kept.
- **A fully transparent, additive composite score** (`recommend/score.py`) — every point in a player's 0–100 rating traces to a named, capped component with its raw inputs attached, rather than a single opaque model output.
- **Exact constrained optimization (MILP) for XI selection**, not a heuristic top-11 sort.
- **A genuine forward-holdout validation**: 149 real 2026 matches (including the entire ICC Women's T20 World Cup) that the shipped models had never seen in any form, scored with zero retraining (`VALIDATION_REPORT.md` §9–§11) — a stronger evidentiary bar than an internal train/test split alone.
- **SHAP-based exact feature attribution** for individual WPA predictions, on top of a plain-language narrative — two distinct explainability layers for two distinct questions ("what does a human need to know" vs. "what exactly drove this number").

Q4. Yo system develop garnu ko main motivation ke thiyo?

The concrete, documented trigger was a real failure of the project's *first* scoring approach: an engine ranking players by **contribution share** (% of team runs/wickets) put a part-time all-rounder **above** a long-proven opener, because contribution share measures how big a player's raw numbers look, not whether those numbers actually moved the team toward winning (`model-training/README.md`, "Why Win-Probability-Added, not contribution-share"). That single wrong ranking motivated building a system whose entire scoring philosophy is "did this player's actions actually change the match's win probability" (WPA), plus a broader motivation recorded in the guiding design principles (`docs/criiq-technical-documentation.md`): selection support should be **transparent** (every number traceable), **honestly evaluated** (always benchmarked against a naive baseline), and built on **real, legally-sourced data** rather than fabricated metrics.

Q5. Tapai ko system traditional selector ko decision-making process bhanda kasari different cha?

A human selector reasons informally and from memory/recent impressions across several scattered sources; Criiq reasons from **explicit, auditable numbers** run through a fixed formula, then hands the *combination* problem to an exact solver rather than intuition:

1. It quantifies "match-winning impact" directly (WPA) instead of relying on memorable-but-noisy raw stats.
2. It never silently drops a player from consideration for reasons a selector might miss — eligibility (`roster.team_pool`), recency, domestic form, and matchup history are recomputed fresh for every single query, point-in-time.
3. XI composition (11 players, ≥5 batting-capable, ≥5 bowling-capable, ≥1 keeper, ≥2 openers) is solved as a genuine 0/1 linear program (`scipy.optimize.milp`), guaranteed to find the mathematically best

combination under those constraints, not "the top 11 scores, patched afterward if that breaks a rule."
4. Every recommendation carries its own reasoning (`recommend/explain.py`) and, on request, an exact SHAP decomposition — a selector can interrogate *why*, not just accept the output.

It does **not** claim to see what a selector sees directly — net-session form, fitness assessments, and tactical/dressing-room judgment have no digital trace and are explicitly outside the data the system can use (`VALIDATION_REPORT.md` §11a's honest read: the AI XI beats random selection by a wide margin but doesn't reproduce the exact real team, precisely because of this gap).

Q6. Tapai ko system ko target users ko hun ra kina?

Coaches and team selectors (the `backend`'s auth model is literally "coach accounts," `auth.py`'s `POST /auth/register/pending-coaches` flow) — people who need to turn "who should play against this specific opponent" into a defensible, explainable decision quickly, across a large pool of eligible players, rather than a fan-facing analytics product. The CLI/report surface (per-player "selector sheets," head-to-head matrices, rotation-XI suggestions for lopsided fixtures) is built around exactly the questions a selector asks before a fixture, not post-match entertainment stats.

Q7. Tapai ko project ko main objectives ke-ke thiye?

Reconstructed from the guiding design principles and the two module purposes (`docs/criiq-technical-documentation.md`):

1. Build a single, trustworthy, self-describing database from legally-sourced, ball-by-ball cricket data (`data-collection` → `criiq.db`).
2. Predict a player's genuine match-winning impact (WPA), not just raw output, with honestly and repeatedly validated models.
3. Turn that prediction, plus matchup/form/standing/risk signals, into one transparent composite score per player.
4. Select a legally-constrained, optimal Playing XI from that score — exactly, not approximately.
5. Explain every recommendation in a way a non-technical selector can act on.
6. Extend the same machinery, cleanly, to a full match simulator and two genetic-algorithm optimizers (bowling rotation, batting order).
7. Validate all of the above against genuinely unseen real matches, not just an internal split, and report results honestly including where they're weak.

Q8. Tapai ko system le available cricket data ra actual player selection practice bich ko gap kasari address garcha?

Two concrete, named gaps are explicitly closed:

- **The "quiet internationally, in-form domestically" gap.** A player can be out of the international XI for a stretch while genuinely in career-best domestic-league form — international-only stats miss this entirely. `player_domestic_form` (data-collection) and the `domestic_form_batting_wpa/domestic_form_bowling_` features (model-training) close it, kept in a *separate* table so domestic-league scale/quality never corrupts the international career line (`ALGORITHMS.md` §0.4).
- **The "who does she actually play for right now" gap.** `players.nationality` was found unreliable (a real wrong-answer case, "ER Oza," is cited directly in `roster_index.py`'s own comment); the system instead derives current team from real match-appearance history, and eligibility (retired/inactive) is recomputed from the complete appearance history rather than a static field.

The system does **not** claim to close the gap between the data it has and everything a real selector uses — net-session form, fitness, and tactical calls are outside its data entirely, and this is stated plainly rather than hidden (`VALIDATION_REPORT.md` §11a).

Q63. CRIIQ ko purpose selectors lai replace garnu ho ki decision-support provide garnu ho?

Decision-support, not replacement — and this is a structural fact about the system, not just a framing choice. The recommendation and simulation outputs are advisory: the backend's `analyze-xi` flow exists specifically to let a coach's *own* hand-picked XI be scored and compared against the AI's optimal pick, reporting the score gap and the exact players that differ (`selection/optimize.analyze_xi`) — a tool for a human to check their own decision against, not a gate that overrides it. `VALIDATION_REPORT.md`'s own summary is explicit about the ceiling: the recommended XI does not reproduce the exact real historical team, "because a real selector uses net-session form, fitness, and tactical calls the model has no access to." The system's honest claim is narrower and more defensible than "replace the selector": it is measurably better than a naive baseline at a well-defined sub-task (ranking players by likely match-winning impact, and packing a legal XI around that ranking), not a claim to see everything a human selector sees.

2. Data Pipeline & Sources

Q9. Tapai ko data sources ke-ke hun ra harek source kina choose garnu bhayo?

Source	Role	Why
Cricsheet (`cricsheet.org`)	Authoritative spine — every ball, lineup, and result	Free, legal, ball-by-ball granularity, machine-readable; the only source in the stack with per-ball detail
Cricsheet people register (`people.csv`)	Player identity join key (`key_cricinfo`)	Lets every external source join **by ID**, not fuzzy name matching
ESPNcricinfo (core JSON API only)	Best-effort metadata: DOB, nationality, role	ESPNcricinfo's *web pages* actively block scraping (verified HTTP 403); the public **core JSON API**, keyed by `key_cricinfo`, is used instead — opt-in (`--enrich`), and the whole pipeline runs to completion without it
Wikidata / Wikipedia	Bowling style, batting hand, squad-page roles/keeper flags	Bowling style/hand aren't in Cricsheet at all; Wikipedia infoboxes are free, legal, and reachable via Wikidata's cricinfo-id sitelinks
ICC rankings (public endpoint or CSV)	"Standing" signal, activity classification	ICC's full list sits behind a credentialed widget; the pipeline only uses the **public** top-10 endpoint when a client ID is configured, and refuses to guess or bypass credentials
RSS feeds (ICC/ESPNcricinfo)	Retirement monitoring	Cheap, low-latency signal — but explicitly **never auto-applied**; only creates a pending alert a human confirms

Deliberately **not** collected, and stated as such rather than faked: ball speed, shot-control%, wagon wheels/preferred shots, pitch/line-length maps — none have a free, legal source for this scope (`data-collection/README.md`, "What is NOT obtainable").

Q10. Multiple data sources bata aayeko data ko quality kasari ensure garnu bhayo?

Layered, not a single check:

1. **Authority is assigned per fact, not per source overall.** Cricsheet wins on *events* (balls, lineups, results); ESPNcricinfo wins on *metadata* (DOB, role, retirement) — so two sources never silently overwrite each other on the same fact.
2. **Deterministic joins, not fuzzy matching**, wherever an ID exists (`key_cricinfo`) — eliminates almost all identity ambiguity by construction rather than by cleanup after the fact.
3. **Explicit validation, not assumed correctness**: `data-collection` replaced the `pandera` dependency with hand-rolled pandas validation (`validate.py`) specifically to keep this checking reliable on the Python version in use.
4. **Real bugs are found by re-checking live output, not just trusting the pipeline** — e.g. the cross-gender identifier collision (Q11), the venue-name conflation bug (two real grounds sharing one

venue_id), and the gender-blended selection-rate denominator (Nepal men's players wrongly read as INCONSISTENT) were all caught by inspecting real numbers on the live database, fixed at the root, and pinned with regression tests — not left as silent inaccuracies.

5. **Nothing is fabricated to fill a gap** — a metric with no legal source is omitted, never approximated, so "missing" is never confused with "wrong."

Q11. Different sources ma player identity resolution kasari handle garnu huncha?

The join chain is deterministic by ID:

```
delivery name --[match info registry]--> cricsheet identifier
              --[people.csv]--> key_cricinfo --> ESPNcricinfo record
```

100% of real players in the live DB resolve to a key_cricinfo — zero fuzzy-name fallback needed for that specific join. Fuzzy matching (rapidfuzz.process.extractOne, WRatio scorer, threshold **88**) exists only as a documented fallback for a few specific, harder joins — e.g. matching externally-researched squad-page names ("First Last") to Cricsheet's "XY Surname" convention, where plain string similarity actually scores *below* the usual threshold, so a dedicated surname+initial heuristic is used instead of forcing fuzzy matching to work (role_finalize.match_full_name_to_cricsheet).

A real identity bug this surfaced and fixed: Cricsheet's own per-match registry reuses the **same identifier across two different real people of different genders** in 32 confirmed cases (e.g. "K Joshi" = a Nepal women's player *and* an unrelated men's player). The schema's UNIQUE(cricsheet_identifier) constraint was relaxed to UNIQUE(cricsheet_identifier, gender), and identity.player_id_for(identifier, gender) mints two distinct, stable player_ids for the ~32 colliding identifiers while leaving the ~99.5% of unambiguous identifiers untouched. Every derived-table builder was audited and fixed to key by (identifier, gender), not identifier alone — this uncovered and removed **2,842 orphaned/blended rows** on rebuild (data-collection/README.md).

Q12. Tapai ko data pipeline ko overall structure kasto cha?

Twelve sequential, idempotent stages orchestrated by criiq_data/refresh.py:

```
1. download_register  -> Cricsheet people.csv (identity registry)
2. download_cricsheet -> competition ZIP archives
3. ingest              -> raw files -> matches / deliveries
4. identity            -> (cricsheet_identifier, gender) -> stable player_id
5. scrape_cricinfo (*) -> optional ESPNcricinfo enrichment
6. styles              -> player_styles (hand, bowling style)
7. rss_monitor         -> retirement_alerts (pending, human-confirmed)
8. rankings/registry/career -> player_registry, player_career, player_cards
9. advanced            -> player_advanced (phase splits, archetype, ...)
10. opponents/impact/venue_stats/matchups/phases -> selector-facing tables
11. ml_features        -> match_features, ball_state (parquet), player_match_impact
12. winprob            -> fits EmpiricalWinProb per (format, gender)
```

A 13th stage, run separately since it depends on stage 12's already-fitted model, computes **WPA** (wpa.build_wpa_for_format → player_match_wpa/player_phase_wpa, then wpa.build_domestic_form_for_forma → player_domestic_form). The result is one self-contained SQLite file (criiq.db, 408 MB, 28 tables, 1,226,642 delivery rows, 5,428 matches at time of writing) plus a handful of Parquet feature files, forming the **entire** hand-off contract to model-training, which reads it strictly read-only and never imports data-collection's own code.

Q13. Data pipeline lai idempotent ra safely re-run garna milne kasari ensure garnu huncha?

Concretely, from `data-collection/README.md`'s "Updated regularly" section:

- ZIP archives are **re-downloaded only when their Last-Modified/Content-Length changes** — a re-run that finds nothing new does no network work.
- Matches already present in the DB are **not re-ingested**.
- ESPNcricinfo enrichment is **re-fetched only for players still missing metadata**, and successful fetches are cached to Parquet.
- Every write path uses **upserts** (`ON CONFLICT DO UPDATE`), not blind inserts, so re-running a stage twice in a row is a no-op on already-correct rows.
- `init_schema(conn)` — which runs the schema, migrations, and a `team_type` backfill — is called at the start of virtually every CLI command, and is itself cheap and idempotent by design (`CODE_WALKTHROUGH.md`).
- A `last_successful_refresh` marker drives a staleness banner (warn > 7 days, stale > 14 days), so a broken schedule is *visible*, not silently stale.

This is what lets `refresh` be pointed at by any scheduler (cron, Windows Task Scheduler, GitHub Actions) without special-casing "first run" vs. "incremental run" — they're the same code path.

Q14. Missing data, especially less-covered players ko case ma, kasari handle garnu huncha?

Three different mechanisms, matched to three different kinds of "missing":

1. **Missing external metadata (style, DOB, keeper flag)** — never blocks the pipeline. A player with no resolved bowling style simply doesn't populate the vs-pace/vs-spin split; a player with no DOB doesn't get an age-advisory narrative line. Coverage is reported honestly (bowling style: 568/1,739 real female T20 bowlers, 32.7%; 868/2,038 male, 42.6%) rather than backfilled with a guess.
2. **Missing/sparse *statistical* history (few matches vs. an opponent, at a venue, or overall)** — handled by **shrinkage**, not by treating a thin sample as ground truth: `shrunk = (n·observed + k·prior) / (n + k)`. A 2-match 100% dismissal rate against one bowler shrinks toward the much more reliable phase-wide baseline rather than being taken at face value (worked example in §4 below).
3. **Missing career history entirely (a debutant, `career_matches < 10`)** — the trained regressor is **skipped outright** (`min_matches_for_model = 10`); a shrinkage-based fallback substitutes, itself scaled down by `confidence = min(1.0, career_matches / 10)` so a 3-match player can post at most ~30% of the full model points. A real, domain-expert-flagged over-rating this fixed: a 3-match Nepal player was posting +12.5 (above a 78-match captain's real +0.2) before this scaling; +3.8 after.

Q15. Recent Form Index bhaneko ke ho ra yeslai kasari calculate garnu huncha?

The **Recent Form Index (RFI)** is a data-collection-side, dashboard-facing form metric (`player_cards`, computed in `stats.py`'s `_rfi`), distinct from model-training's separate `form_*` regression features (Q21) though built on the same idea. It is an **exponentially recency-weighted average of a batter's last `form_window_innings` scores**: the *i*-th-most-recent innings gets weight `lam**i` (`lam = SETTINGS.rfi_lambda`, less than 1, so more recent innings count more), and the index is the weighted mean `sum(w·r) / sum(w)`, not a plain average — deliberately recency-biased so a player's *current* hot or cold streak shows up in the number, rather than being diluted by an old purple patch (`data-collection/CODE_WALKTHROUGH.md`).

3. Feature Engineering & Data Leakage

Q16. Feature engineering garda data leakage kasari prevent garnu huncha?

Three concrete mechanisms (`ALGORITHMS.md` §3.7, `docs/criiq-technical-documentation.md` "How leakage is prevented"):

1. **Point-in-time reconstruction, not aggregate-table reuse.** The database's own aggregate tables (`player_career`, `player_vs_opponent`, …) include every match, including the one being predicted and every one after it — so the training builder (`features/historical.py`) never reads them directly. It rebuilds every feature as-of each match from per-match rows: sort by date, `.shift(1)`, *then* `.expanding()/.rolling()` — so a match's own row can never enter its own features. Unit tests assert this literally: row *N*'s career mean must equal the mean of rows 1..*N*-1 only.
2. **Temporal splits, never row-level splits, for evaluation.** Rows from the same match are correlated (teammates share a result), so a random row split would leak outcome information across the train/test boundary. The split is always by **match-date quantile** — everything before the 80th-percentile cutoff date trains, everything on/after it is held out.
3. **A deliberate non-feature.** The batter-vs-bowler `dismissal_matchup` table is a career *total* with no per-match breakdown, so it cannot honestly be reconstructed "as of" a past match. Rather than accept the leakage, it is excluded from `MODEL_FEATURES` entirely and used only as a transparent, rule-based scoring component (`matchup_part`) at live-scoring time, where using full history is correct (a live recommendation is genuinely about a future match).

Cross-stream joins (attaching domestic-league form onto an international row) use `pd.merge_asof(direction="backward` specifically because a normal `groupby().transform()` assumes the joined rows belong to the same averaging group — domestic and international matches are two separate event streams, so the "most recent prior record in another timeline" join (the same technique finance uses for "most recent stock price as of this trade's timestamp") is the correct tool.

4. Win-Probability-Added (WPA) — the Core Modeling Idea

Q17. WPA bhaneko ke ho ra raw batting/bowling statistics ko satta WPA kina use garnu bhayo?

WPA (Win-Probability-Added) measures whether a player's actions actually moved their team toward winning, not how large their raw numbers looked. It replaced an earlier **contribution-share** target (% of team runs/wickets) after a real, documented failure: contribution share ranked a **part-time all-rounder above a long-proven opener**, because a batter who scores 40 off 25 in an already-decided blowout "contributes" a lot by raw count but barely moves the result, while a tense 20 in a close chase might contribute far less by raw count yet be the actual reason the team won.

The concrete evidence that triggered the switch: **Smriti Mandhana averages +0.077 batting WPA per match across 144 matches; the player who had wrongly outranked her under contribution-share averaged −0.002 across 23 matches** — despite that player's raw "contribution" numbers looking competitive (`ALGORITHMS.md` §3.2). This is the same idea as a basketball player's win-probability-added versus their raw point total: a garbage-time three-pointer in a 30-point blowout counts identically to a game-winning buzzer-beater in the box score, but the two are worlds apart in actual impact.

Q18. WPA ko mathematical formula formulate garnus ra harek term explain garnus.

For every ball `t` in a match, the already-fitted win-probability model (§4.4 of `ALGORITHMS.md`, `Empirical-WinProb`) outputs `win_prob_batting` — the batting side's probability of winning **given the match state right after that ball** (score, wickets, overs used/remaining, target if chasing). WPA is the *change* in that probability caused by one ball:

```
delta_wp(ball_t) = win_prob_batting(state after ball_t) − win_prob_batting(state after ball_{t−1})
```

(the very first ball of an innings uses the true starting-state win probability instead of an undefined "ball 0.")

Attribution: the batter on strike is credited `+delta_wp` toward their per-match **batting WPA**; the bowler is credited `−delta_wp` toward their **bowling WPA** (a ball that helps the batting side necessarily hurts the bowling side's win chance, so the sign simply flips). The non-striker gets nothing.

Worked example: if the batting side's win probability is 42% before a ball and 47% after a six, `delta_wp = +0.05`. The striker's batting WPA for that ball is `+0.05`; the bowler's bowling WPA for that ball is `−0.05`. Summed across a match (or averaged across many matches), this becomes the per-match/career WPA figure the regression models (Q21) are trained to *predict in advance* from historical stats, for matches not yet played.

Q19. Per-ball win probability kasari calculate garnu huncha?

`EmpiricalWinProb` (`data-collection/criiq_data/winprob.py`) is **empirical, not parametric**: it bins the continuous match state into discrete buckets rather than fitting a smooth curve, because there isn't enough data density to fit a smooth function reliably, and real T20 win-probability curves aren't well-approximated by a simple textbook shape anyway.

Binning: one over per bin for balls bowled/remaining; wickets clipped to `[0, 10]`; for a chase, runs-required in bands of 5 (capped at 30 bands); for setting a total, score in bands of 10. This gives two kinds of keys: `(balls_remaining // 6, wickets, runs_required // 5)` when chasing, and `(balls_bowled // 6, wickets, score // 10)` when setting.

Fitting — shrunk empirical win rate per bin:

```
wp(bin) = (Σ y_bin + α · prior) / (count_bin + α)
```

where `y` (in `{0,1}`) is whether the batting side eventually won, `prior` is the overall raw win rate for that situation type (chase/set), and `α = winprob_shrinkage = 12.0`. Bins with little real data get pulled

strongly toward the sensible overall prior; bins with abundant data are trusted almost entirely on their own empirical rate — the same shrinkage family used everywhere else in the project.

Backoff for rare states: four grids are actually fit (fine and coarse, for chasing and setting separately). A lookup first tries the fine grid; if that exact bin is too sparse, it backs off to the coarser grid; if even that's missing, it falls back to the raw prior — a sensible answer always exists, even for a state like "9 down needing 40 off 3 balls."

Scope and evaluation: one model per (`format, gender`), never blended (men's and women's scoring patterns differ enough that pooling would bias both). Held out **by match, not by ball** (`zlib.crc32`-hashed match IDs, ~20% of matches held out whole), since consecutive balls in one match are correlated. Real results: female Brier 0.1557 vs. a 0.2447 constant-prior baseline (93,714 test balls); male Brier 0.1727 vs. 0.25 baseline (145,372 test balls).

Q20. Batting ra bowling WPA lai separate targets ko rupma kina train garnu bhayo?

So that a specialist's structural zero in their *other* discipline never drags down the rating for the discipline they actually play. If a single blended "overall impact" label were used, a pure bowler's near-zero batting involvement would pull that combined number down even though it says nothing about their bowling. This was checked directly, not assumed: a single-label learning-to-rank alternative (`LGBMRanker`, Q23) that collapses batting+bowling WPA into one quantile-based relevance grade lost decisively to the two-independent-regressor approach on real data (top-11 hit-rate 0.626 vs. 0.579 regression-vs-ranker; Spearman 0.28 vs. 0.116) — the project's own documented explanation is exactly this "averaging out role-specific signal" effect.

Q21. Tapai ko WPA regression model le kun-kun features use garcha?

22 numeric features + 1 categorical (`features/schema.py`, `MODEL_FEATURES`), shared identically between the training builder and the live-scoring builder so train/serve never drift apart:

Group	Features
Career-to-date (expanding, pre-match)	`career_matches`, `career_bat_contribution_pct`, `career_bowl_contribution_pct`, `career_bat_sr`, `career_bowl_economy`, `career_batting_wpa`, `career_bowling_wpa`, `career_bowl_wickets_per_match`
Recent form (15-match window, geometric decay 0.9)	`form_bat_contribution_pct`, `form_bowl_contribution_pct`, `form_bat_sr`, `form_bowl_economy`, `form_batting_wpa`, `form_bowling_wpa`, `form_bowl_wickets_per_match`
Versus this opponent (expanding, shrunk)	`vs_opp_matches`, `vs_opp_bat_contribution_pct`, `vs_opp_bowl_contribution_pct`, `vs_opp_win_pct`
At this venue (expanding, shrunk)	`venue_matches`, `venue_bat_contribution_pct`, `venue_bowl_contribution_pct`
Domestic form (bridged via `merge_asof`, shrunk, sample capped at 10 effective matches)	`domestic_form_matches`, `domestic_form_batting_wpa`, `domestic_form_bowling_wpa`
Categorical (one-hot)	`role_bucket` (batter / bowler / all-rounder / keeper)

Targets: `batting_wpa`, `bowling_wpa`. `career_bowl_wickets_per_match` is masked to matches where `balls_bowled > 0` before averaging, since the source table stores `0` (not `NULL`) for a pure batter's non-bowling matches — without the mask the feature dilutes toward zero for anyone who mostly bats; fixing this alone measurably improved bowling-WPA correlation (~0.09 → ~0.10).

Explicitly tried and rejected, on real backtest evidence: `bowl_dot_pct` / `bowl_boundary_pct` / `bowl_strike_rate` (bowling containment metrics — made bowling prediction *worse*), `age_years` (helped batting negligibly, measurably *hurt* men's targets), `bowl_style_bucket` (no measurable difference). All three are still computed and shown as narrative color in `explain` output, just never fed back into a prediction.

Q22. HistGradientBoostingRegressor nai kina choose garnu bhayo?

Worth correcting the premise directly, since it's not quite what's shipped: the **production `batting_wpa` model is a regularized `RandomForestRegressor`** (`n_estimators=300, max_depth=10, min_samples_leaf=5`, plus `max_features="sqrt"` for the female scope only). `HistGradientBoostingRegressor` appears only as **one half of the `bowling_wpa` model**, inside a `VotingRegressor` that blends it with a second, more conservative `RandomForestRegressor` (`weights=[0.7, 0.3]`, favoring the boosted half).

Why HistGB specifically for that half, and depth-limited: an unconstrained `HistGradientBoostingRegressor` (sklearn defaults) was found to **memorize per-match noise** on bowling-WPA, the weaker of the two targets — capping `max_depth=2` (a tree that can only ask 2 sequential questions) raised mean correlation from ≈0.088 to ≈0.105 *and* roughly halved run-to-run variance (std ≈0.005 → ≈0.002), confirmed across a 3-fraction × 5-seed repeated holdout, not a single lucky split (`ALGORITHMS.md` §3.3). Gradient boosting was chosen for this half specifically because it corrects the *residual errors* of prior trees — useful for squeezing out a thin signal — while the paired Random Forest (which averages independently-trained trees) supplies a different, less-correlated kind of error to average against (ensembling, Q23/§5.5).

Q23. Different machine learning models lai kasari compare garnu bhayo?

Via a **repeated-holdout comparison notebook** (`model_comparison_robust.ipynb`, 223 result rows), not an automated hyperparameter search — no `GridSearchCV`, `RandomizedSearchCV`, or Optuna exists anywhere in the codebase. The protocol, applied identically to every model-family and hyperparameter decision:

- **3 different temporal-holdout fractions** (0.15 / 0.20 / 0.25) — guards against a single lucky/unlucky split date.
- **5 random seeds per fraction** for every stochastic model — guards against a single lucky/unlucky seed.
- Metrics (MAE, R^2, Pearson correlation) reported as **mean ± standard deviation** across all 15 runs, and every *apparent* winner re-checked separately on **both** gender scopes before being adopted (since one hyperparameter, `max_features="sqrt"`, was found to help female but not transfer to male — see below).

Candidates evaluated per target: mean-predictor baseline, the actual production model (imported directly from `train.py`, so no configuration drift), Linear/Ridge regression, a generic Random Forest, `HistGradientBoosting` (sklearn defaults), LightGBM, and XGBoost (default and tuned). On batting WPA the production Random Forest essentially tied the top-scoring generic RF (the gap is seed noise) and beat every gradient-boosting/linear alternative on correlation; on bowling WPA the production `VotingRegressor` won outright on correlation with the *lowest* run-to-run variance of any candidate (`model-training/README.md`, "Models tried — the full comparison").

Q24. Temporal holdout bhaneko ke ho ra random train-test split ko satta yeslai kina use garnu bhayo?

A **temporal holdout** splits by **match date**, never by row: sort all match dates, take the cutoff at the (1 − `backtest_test_fraction`) quantile — by default the 80th percentile — everything strictly before that date trains the model, everything on/after it is held out for scoring. This directly mimics real deployment: predicting future matches from past ones.

A **random row-level split** would be actively wrong here for a specific, concrete reason: rows from the same match are statistically correlated (a batter's WPA in a given match is entangled with their own team's bowlers' WPA in that same match, since they share a result). A random split would let some rows from one match land in training while other rows from the *same* match land in the test set — leaking real signal about the outcome across the train/test boundary and inflating the reported accuracy relative to what a genuinely future prediction would achieve. Standard k-fold cross-validation has the identical problem for the identical reason, which is why it is deliberately not used (`docs/criiq-technical-documentation.md`, "Validation strategy").

Q25. Tapai ko R^2 values 0.015 bhanda tala chan. Yeslai kasari justify garnu huncha?

Honestly, by what R^2 actually measures here and what the alternative (correlation) measures instead, rather than by explaining it away. R^2 near zero (measured: 0.0175 for batting, 0.0091 for bowling on the production models, `model-training/README.md`) means the models explain almost none of the *variance* in a single match's raw WPA number — and that is expected, not a bug: per-match cricket WPA is dominated by high-variance, near-unpredictable events (one dropped catch, one hat-trick, one hot 25-ball innings) that no pre-match feature can see coming.

Critically, **the mean-predictor baseline's MAE is also barely beaten** (sometimes technically *not* beaten) by the trained models — a genuinely honest report, not hidden. But R^2/MAE answer "how precisely does this predict one match's exact number," which is **not the question the system actually needs answered**. The task is *ranking* players relative to each other for a selection decision, which is what **Pearson correlation** (0.122 batting-female, 0.093 batting-male, 0.076 bowling-female, 0.073 bowling-male — current backtest, `ALGORITHMS.md` §3.5) and **top-quartile hit-rate** (30–36% vs. a 25% random baseline) directly measure, and both are clearly, consistently above chance across every re-run and every holdout fraction tested (§3 of `ALGORITHMS.md`, three fractions, all within a ±0.02 band). Low R^2 with meaningfully-positive, stable correlation is the textbook signature of "a genuinely noisy target with a real but modest ranking signal" — exactly what this project reports it has, not more.

Q26. Pearson correlation R^2 bhanda tapai ko use case ma kina important cha?

Because the system's actual job is **ranking players relative to each other within one match's candidate pool**, not predicting any single player's exact WPA number. R^2/MAE measure absolute error against the true value — useful for "how close was this one prediction," largely irrelevant to "did the model correctly identify who the better performers were." Pearson correlation measures whether predicted and actual values move together — i.e., whether the model's *ordering* of players tracks the real ordering — which is precisely what feeds into `select_xi`'s objective. The project found these two metrics can even disagree in an instructive way: the trivial mean-predictor baseline has the *lowest* MAE for both targets, yet its correlation with anything is exactly 0 by construction (it produces the same number for everyone) — the strongest possible illustration that MAE alone would make a useless model look "best" (`ALGORITHMS.md` §3.5).

Q27. Brier score bhaneko ke ho ra yeslai kina use garnu huncha?

Brier score is the mean squared error of a *probability* forecast against the actual binary outcome:

```
Brier score = mean( (p − y)² )
```

where p is a predicted probability (e.g. "70% chance the batting side wins") and y (in {0,1}) is the real outcome. Lower is better. It is used (alongside log-loss) to evaluate EmpiricalWinProb, because the win-probability model's whole job is producing a *calibrated* probability, not just picking the more-likely side — a model that says "70%" needs to actually be right about 70% of the time across many such calls, not merely be directionally correct more often than not. This is exactly the same standard used to score weather forecasters' "70% chance of rain" calls. Real measured results: female Brier 0.1557 vs. a 0.2447 naive constant-prior baseline; male Brier 0.1727 vs. 0.25 baseline — both meaningfully better (lower) than always guessing the overall average win rate, held out by whole match (never by ball, since consecutive balls in a match are correlated).

Q28. Sign agreement accuracy le ke measure garcha ra yo kina important cha?

Sign-agreement accuracy asks the coarsest possible question about a prediction: among players with a genuinely nonzero *actual* WPA, did the model at least get the **direction** right — positive (helped the team win) vs. negative (hurt it) — regardless of magnitude? Measured: 52–55% across both genders and both targets (VALIDATION_REPORT.md §10B) — just above a 50% coin flip. It's reported for completeness, not oversold: the project is explicit that this task was never framed as binary classification ("good performance vs. bad"), it's a continuous *ranking* signal, and the fairer read of the same underlying model is the top-quartile hit-rate (30–36% vs. a 25% random baseline, §2) — which is meaningfully above chance even while raw sign-agreement sits close to it. The honest reason both numbers can be true at once: per-match WPA has enough noise that "was the sign even right" is a genuinely hard bar for a single match, while "did the model correctly flag who was *among the best* that match" is an easier, more forgiving, and more decision-relevant question — and the model does noticeably better on the latter.

Q29. A few matches bhayeka players ko lagi shrinkage ko role ke ho?

Shrinkage is the mechanism that stops a tiny, noisy sample from being trusted as if it were a large, reliable one. General formula (used identically for opponent/venue/domestic-form features, batter-vs-bowler dismissal rates in the simulator, and win-probability bins):

```
shrunk_estimate = (n · observed + k · prior) / (n + k)
```

- observed — the specific, possibly-sparse estimate (e.g. this batter's dismissal rate against this exact bowler).
- n — how many real observations back it.
- prior — the broader, reliable fallback (e.g. the phase-wide baseline rate).
- k — "prior strength": how many real observations it would take to trust the specific number as much as the prior (shrinkage_prior_matches = 8 for opponent/venue features; winprob_shrinkage = 12.0 for win-probability bins).

Worked example: a batter dismissed twice in 2 balls against one bowler (observed = 1.0, n = 2) against a 6% phase-wide baseline (prior = 0.06, k = 8): shrunk = (2×1.0 + 8×0.06) / 10 = 0.248 — elevated above the 6% baseline (the sparse evidence isn't ignored) but nowhere near a naive "100% dismissal rate" claim from 2 data points. As n grows, the formula automatically trusts the observed rate more. This is mathematically the same family as the classical **Beta-Binomial Bayesian conjugate update** and is related to the **James-Stein estimator** result (blending several small-sample estimates toward a common average produces *more accurate* estimates on average than trusting each one in isolation) — a simpler, directly-interpretable formula standing in for full Bayesian machinery.

For a player specifically, the identical logic gates the trained model itself: below min_matches_for_model = 10 career matches, the regressor is skipped and a shrinkage-based fallback (career WPA shrunk toward the scope-wide prior) substitutes, additionally scaled by career_matches / 10 so a 3-match player is capped near 30% of full model points.

Q30. Predicted vs. actual WPA plot lai kasari interpret garnu huncha?

`VALIDATION_REPORT.md` §7 gives two real, concrete examples rather than an abstract scatterplot description. For India vs. Sri Lanka (2025-12-21, female): JI Rodrigues scored an actual batting WPA of **+0.280** but was predicted **+0.020**; H Kaur actual **+0.174** vs. predicted **+0.034**; S Mandhana actual **+0.102** vs. predicted **+0.076** (much closer); Shafali Verma actual **−0.162** vs. predicted **+0.049** (wrong sign). The honest reading the report itself gives: the model gets the **direction** right for steady, high-volume contributors (Mandhana, whose prediction sits close to the diagonal `y = x` line a perfect model would trace) but **badly underestimates rare, standout one-off performances** (Rodrigues' 69, a men's-example Owen's match-winning 50) — points that sit far below the `y = x` line on the actual-WPA axis relative to their predicted value. That's the visual signature of a model that has learned a sensible *central tendency* from history but structurally cannot foresee a rare breakout innings from pre-match form alone — exactly the documented limitation that per-match cricket variance is dominated by events the model has no leading indicator for.

Q31. Residual plots le model ko bias ko barema ke information dincha?

The same §7 examples double as the residual story: `residual = actual − predicted` is consistently **positive and large** for standout performances (Rodrigues: `0.280 − 0.020 = +0.260`; Owen: `0.568 − (−0.002) = +0.570`) and small/near-zero for steady contributors (Mandhana: `0.102 − 0.076 = +0.026`). A residual plot (predicted on the x-axis, residual on the y-axis) would show this as **residuals fanning out and skewing positive at the high end of actual WPA** rather than scattering randomly around zero — the signature of a model that systematically **underestimates its own tail** (rare big innings), not one with a simple additive bias across the board. This matches the project's own stated reasoning for winsorizing training labels at the 95th/98th percentile (Q on winsorizing, if asked) — extreme, rare outcomes are capped during *training* specifically because an unclipped fit would otherwise be dominated by chasing those tail events at the expense of the much more common, steadier cases, while *evaluation* is always run against the raw, unclipped actual values so the reported residual pattern above is genuine, not hidden by the clip.

5. The Composite Recommendation Score

Q32. Tapai ko composite recommendation score ko seven components ke-ke hun?

`RecommendationScorer` (`recommend/score.py`) produces a 0–100 score as a documented sum of parts:

Component	Range	What it rewards
`model_part`	0 to 45	Predicted batting + bowling WPA (or a shrinkage fallback below 10 career matches), scaled by `wpa_scale = 450`
`matchup_part`	−21 to +21	Aggregate dismissal history vs. this opponent's bowlers (±8) + style-specific pace/spin edge (±5) + symmetric bowling-side record against the opponent's specific batters (±8)
`phase_fit_part`	0 to 8	In-team percentile/rank in the player's best batting or bowling phase (powerplay/middle/death)
`standing_part`	0 to 20	The separate, transparent `importance_score` formula, plus an ICC top-10 bonus
`risk_part`	≤ 0	Age advisory (−3), INCONSISTENT selection history (−5), recalled-from-INACTIVE (−4)
`strike_rate_part`	≤ 0	Soft penalty (cap −6) for a below-par career strike rate, offset by a game-awareness factor
`team_need_part`	0 to 5	Pool-level synergy bonus, added after individual scoring in `score_many()`

Q33. Model output matra use garnu ko satta composite score kina use garnu bhayo?

Because the trained WPA model, on its own, is a genuinely weak, noisy signal (R^2 near zero, correlation 0.07–0.12 — Q25) that says nothing at all about several things a real selection decision needs: whether this specific player has a favorable matchup against this specific opponent, whether they specialize in a phase this team's pool is thin in, whether their selection history or age is a red flag, or whether their raw strike rate is a genuine liability. The composite score's other six components carry exactly that information, each from a transparent, independently checkable source (real dismissal history, real phase splits, the hand-specified `importance_score`, the registry's activity classification) — so the WPA model contributes what it's actually good at (a ranking signal net of everything else) while the rest of the formula supplies what a single learned number structurally cannot.

Q34. Composite score ko weights kasari determine garnu bhayo?

Calibrated against real observed distributions and validated by inspecting real output, not fitted as fully free hyperparameters and not guessed blind. Concretely: `wpa_scale = 450` was chosen so that a genuinely elite average WPA (≈0.09, near the real 99th percentile of players with 10+ matches) lands close to, but doesn't exceed, the 45-point cap on `model_part`; the team-need scale is calibrated against the real standard deviation of combined phase-WPA (≈0.065–0.078) so that "one standard deviation above

benchmark" reliably reads as a noticeable strength. A real, documented correction: the *first* team-need scale (×200, later ×35) was roughly 6× too aggressive and hit its point cap for nearly every player in a pool regardless of actual relevance — caught not by a metric, but by a maintainer eyeballing real output and finding it implausible (`ALGORITHMS.md` §5.4). This "calibrated, then verified by inspection" methodology is applied consistently and is documented as a real limitation too (§ "Known limitations" below): these constants are not sensitivity-tested or automatically tuned, unlike the model hyperparameters, which *are* chosen from a repeated-holdout search.

Q35. Matchup component bhaneko ke ho ra yeslai kasari calculate garnu huncha?

`matchup_part` has three sub-signals, each independently capped:

1. **Aggregate batting matchup (±8):** the batter's real, shrunk dismissal history against this opponent's specific bowlers (`dismissal_matchup`, minimum 6 balls per pairing to count as a real sample).
2. **Style-specific edge (±5, secondary/backup signal):** the opponent's actual current pace/spin mix (`player_styles.bowling_style`) weighted against the batter's own vs-pace/vs-spin dismissal record, itself shrunk with a strong prior (`strength = 48`, since this split is data-sparse) — falls back to contributing exactly 0 if coverage is insufficient at any step, never guesses.
3. **Bowling-side matchup (±8, symmetric polarity):** the bowler's real wicket-rate record against this opponent's specific batters — a fix for a structural gap where specialist bowlers used to get almost no matchup credit at all.

When a coach names the opponent's actual XI (`--opponent-players`), every one of these three scopes to exactly those 11 players (by real ID, not team-wide averages) — this is the "counter-optimization" the project describes: because the MILP maximizes total score and the matchup inputs are now scoped to the *specific* named opposition, the recommended team naturally tilts toward players who counter it, with no separate objective needed.

Q36. Need component bhaneko ke ho ra yesle team balance kasari improve garcha?

`team_need_part` (`recommend/team_need.py`) answers a question no single player's score can answer alone: "is this specific candidate pool collectively weak in some phase of the game?" It finds the **one** phase (powerplay/middle/death) where the pool's average combined phase-WPA falls furthest below a format-and-gender-scoped benchmark, then rewards whoever is genuinely strong in exactly that phase — up to +5, **never a penalty** for not being the answer. This is applied after individual scoring, in `score_many()`, since it's a property of the whole pool, not any one player. A real bug this fixed: the benchmark used to blend both genders' `player_phase_wpa` into one number, so a men's pool's "are we weak at death bowling" question was silently answered against a benchmark partly built from women's data; real, gender-scoped benchmarks now measurably differ (e.g. death-overs benchmark ≈−0.002 for men vs. ≈+0.003–0.005 for women on live data).

Q37. Risk penalties ke-ke hun ra kahile apply hunchan?

Three penalties, all sourced from `player_registry` and all **penalties only** (never a bonus): age advisory (**−3**, role-specific age threshold — 38 for fast bowlers, 40 for others), INCONSISTENT selection history (**−5**, an in-and-out fringe player per the registry's recency-and-regularity classification), and recalled-from-INACTIVE (**−4**, a player absent 3+ years who's being given a second look via a genuine recall, not an automatic override). These fire from the same `player_registry` eligibility engine that also *excludes* RETIRED players outright and hard-excludes plain INACTIVE (non-recalled) players from the candidate pool entirely — `risk_part` only ever applies to players who cleared eligibility in the first place, as a softer signal on top.

Q38. Phase component bhaneko ke ho ra player ko best phase kasari identify garnu huncha?

`phase_fit_part` (0–8 points) rewards being a genuine **phase specialist** — powerplay (overs 1–6), middle (7–15), or death (16–20) — using `player_phase`'s real, data-derived **in-team percentile/rank** for whichever of a player's batting or bowling phase splits is strongest, not an absolute cross-league bar. Separately (and not part of the score itself, since the composite is win-probability-driven, not raw hitting), `selection/optimize.role_highlights` identifies the single best **finisher / power hitter / anchor** across the *whole* scored pool, ranked by that role's own defining metric (finisher: death-overs strike rate; power hitter: six rate) rather than the composite score — this is the deliberate fix for the recurring "she's our best finisher but isn't recommended" case (real example: RM Ghosh, death strike rate 166, 45 career sixes, composite rating only 7.8/100, still correctly surfaces as the pool's best finisher).

Q39. WPA model component lai 0–45 score ma kasari normalize garnu huncha?

```
active_wpa = []  # only disciplines this player's role supports
if batting_capable: active_wpa.append(predicted_batting_wpa)
if bowling_capable: active_wpa.append(predicted_bowling_wpa)
active_wpa.sort(reverse=True)

primary = active_wpa[0] if active_wpa else 0.0
secondary_bonus = sum(active_wpa[1:]) * secondary_discipline_weight  # 0.4
model_part = min(45.0, (primary + secondary_bonus) * wpa_scale)      # wpa_scale = 450.0

if not model_applied:  # cold start
    model_part *= min(1.0, career_matches / min_matches_for_model)
```

A pure specialist's `model_part` comes entirely from their one relevant discipline at full weight; an all-rounder's comes from their **stronger** discipline at full weight plus their weaker one at only **40%** weight (`secondary_discipline_weight = 0.4`) — rewarding genuine dual-skill value without letting it fully double-count against a pure specialist. `wpa_scale = 450` is calibrated (not guessed) against the real distribution of per-match WPA among established players (median ≈0, 95th percentile ≈+0.046, 99th ≈+0.086), so a genuinely elite average WPA lands close to, without exceeding, the 45-point cap.

Q40. Composite score le raw WPA model bhanda better selection garcha bhanera kasari test garnu bhayo?

Two independent, real-data checks, both documented with actual numbers rather than asserted:

1. **`VALIDATION_REPORT.md §11a`** — the full composite-score-plus-MILP pipeline (not the raw model alone) was compared against a random-11-from-the-same-pool baseline on 142 real historical team selections from the 149-match genuine forward holdout: **55.8%** average overlap with the real historical XI vs. **37.2%** for random selection — a wide, consistent margin, confirming the full scoring formula (not just the WPA model) is picking genuinely good players.
2. **A specific sanity check that motivated keeping WPA at all as the model's target** (Q17): Mandhana (+0.077 average WPA, 144 matches) vs. the previously-mis-ranked player (−0.002, 23 matches) under the old contribution-share target — the composite formula built around WPA correctly separates these two, where the old formula didn't.

The engine (`selection/optimize.analyze_xi`) also lets a coach's own hand-picked XI be directly diffed against the provably-optimal MILP output on the *same* composite score, reporting the exact score gap and which players differ — a standing, re-runnable check that the composite-plus-MILP combination genuinely outperforms an arbitrary manual pick on its own terms, not just against a random baseline.

6. Playing XI Selection — Mixed-Integer Linear Programming (MILP)

Q41. XI selection problem lai mathematically kasari formulate garnu huncha?

As a **0/1 Mixed-Integer Linear Program**:

```
maximize      Σ score_i · x_i                (a linear objective)
subject to    Σ x_i = 11                      (exactly 11 players)
              Σ (is_batting_capable_i) · x_i ≥ 5
              Σ (is_bowling_capable_i) · x_i ≥ 5
              Σ (is_keeper_i) · x_i ≥ 1        (soft — degrades if no keeper data)
              Σ (is_opener_i) · x_i ≥ 2        (soft — same degradation)
              x_i in {0, 1}  for every candidate i
```

`x_i` is a binary "select player *i*" decision variable per pool member, `score_i` is that player's already-computed composite score (Q32), and everything on the left-hand side of every constraint is a linear sum of binary variables — genuinely a linear program with integer restrictions, not an approximation of one. Solved by `scipy.optimize.milp`, which uses **branch-and-bound**: solve the relaxed version (allow `x_i in [0,1]`, a much easier plain LP), and if the answer isn't already all-integer, branch on a fractional variable (force it to 0, force it to 1) and recurse, pruning branches that can't beat the best whole-number solution already found. This is a **complete** search — it provably returns the true global optimum, or proves no feasible solution exists, never an approximation.

Q42. Top 11 scores matra select garnu ko satta MILP kina use garnu bhayo?

Because "take the top 11 by score" can trivially violate a hard composition constraint, and there's no guaranteed cheap patch-up once it does. Worked illustration from `ALGORITHMS.md` §1: with 4 candidates (scores 9, 8, 7, 6; only the middle two batting-capable) and a "need ≥2 batting-capable" rule, greedy-by-score picks the top two (9+6=15, only 1 batting-capable) and then has to awkwardly patch the choice, while the MILP simply *knows* the whole feasible region up front and finds the true best legal combination (8+7=15, the only feasible pair with 2 batting-capable players) in one solve, with no fix-up step. More generally: **if a problem can be written exactly as a linear objective plus linear constraints, solve it exactly rather than approximate what's directly computable** — a greedy heuristic or even a genetic algorithm could search this same space, but only ever produce an approximate, possibly-suboptimal answer to a problem that already has an exact, efficiently-computable one.

Q43. Legal playing XI ko lagi kun-kun constraints enforce garnu huncha?

Exactly 11 players (`xi_size = 11`); at least 5 batting-capable (`min_batting_capable = 5`); at least 5 bowling-capable (`min_bowling_capable = 5`); at least 1 recognized wicket-keeper where keeper data exists for the pool (`min_keepers = 1`, soft); at least 2 recognized openers by real historical batting position, not guesswork (`min_openers = 2`, soft). All are configured in `criiq_models/config.py`'s frozen `Settings` dataclass, not hardcoded inline.

Q44. Insufficient qualifying players, Bhayeko pool lai kasari handle garnu huncha?

Constraints degrade gracefully rather than making the problem infeasible. `min_keepers`/`min_openers` are computed as `min(setting, available_in_pool)` — if a team's pool has zero recorded keepers, the keeper requirement silently drops to 0 for that specific solve rather than the MILP failing outright purely because of a data gap. This is a deliberate design choice: in production, "the solver returns no answer at all because of missing metadata" is judged a worse failure mode than "temporarily not enforcing a soft constraint the data can't currently support." (`min_batting_capable` and `min_bowling_capable`, the two hard structural constraints, are not softened the same way, since a legal cricket XI genuinely needs both disciplines represented.)

Q45. MILP solver fail bhayo bhane ke huncha?

If `scipy.optimize.milp` still fails to return a solution (constraint softening notwithstanding), `_greedy_fallback` takes over: start from the top-11 players by raw score, then for any constraint that XI still violates, swap in the cheapest fix — the lowest-score change that restores compliance. This is explicitly **not claimed to be optimal**, only guaranteed to return *some* legal XI so the system never returns nothing to a coach mid-request.

Q46. Greedy repair fallback bhaneko ke ho ra kahile use huncha?

Two distinct "greedy repair" mechanisms exist in the codebase, for two different subsystems, worth distinguishing:

1. **`selection/optimize.py`'s `_greedy_fallback`** (Q45) — the MILP's own last-resort fallback, used only if the exact solver itself fails to return a feasible answer.
2. **`simulate/ga_bowling.py`'s `_repair`** — a different, *routinely* used mechanism inside the bowling-rotation genetic algorithm: crossover/mutation will regularly produce an illegal 20-over rotation (a bowler assigned over 4 overs, or two identical overs adjacent). Rather than penalizing bad candidates in the fitness score (wasteful — real simulator calls would be spent on candidates that get discarded anyway) or rejecting them outright (which shrinks the population), `_repair` deterministically fixes every child: count each bowler's overs, cap at 4, reassign the overage to bowlers with spare capacity (preferring recognized specialists), then re-sequence with a greedy no-repeat-adjacent rule. This runs on essentially every generation, not just as an emergency fallback.

Q47. MILP solver ko correctness kasari test garnu huncha?

By a dedicated pytest suite (`tests/test_selection.py`) asserting exact, checkable properties on synthetic pools, not just "it runs":

- `test_select_xi_respects_role_quotas_and_maximizes_score` — the chosen XI both satisfies every quota **and** is the true score-maximizing legal combination.
- `test_select_xi_forces_in_a_keeper_when_one_exists_in_the_pool` / `test_select_xi_skips_keeper_co` — the soft keeper constraint activates and degrades correctly.
- `test_select_xi_finds_keeper_even_outside_the_display_pool_cutoff` / `test_select_xi_forces_in_op` — regression tests for a real, previously-fixed bug where a required keeper/opener ranked below the top-20 display shortlist and was wrongly excluded before the MILP even saw them; these confirm the MILP always runs over the **full** pool.
- `test_select_xi_handles_empty_pool` — a degenerate-input guard.
- `test_analyze_xi_matches_optimal_reports_zero_gap` / `test_analyze_xi_flags_violations_and_scores` — `analyze_xi` (a coach's own hand-picked XI) is checked against the *same* solver's true optimum, confirming the reported score gap and constraint violations are correct.
- `test_select_xi_orders_output_like_a_team_sheet_not_by_score` — the presentation layer (openers in slots 1–2, bowlers at the tail) is verified independently of the selection logic itself.

Q48. All-rounders jasto multiple roles fulfill garna sakne players lai kasari handle garnu huncha?

Two separate mechanisms, at two different layers:

1. **In scoring** (Q39): an all-rounder's `model_part` uses their *stronger* discipline at full weight and their weaker one at only 40% weight (`secondary_discipline_weight`), so genuine dual-skill value is rewarded without fully double-counting against a pure specialist with the same peak skill.
2. **In XI construction** (Q41/Q43): the MILP's role constraints are $\Sigma\ x_i \geq 5$ for both batting-capable and bowling-capable, using **inclusive**, non-exclusive role-capability flags per player — an all-rounder who is flagged both batting-capable and bowling-capable can count toward *either* quota simultaneously,

which is exactly what lets a genuine all-rounder's selection relax pressure on both quotas at once (a real practical benefit of picking a strong all-rounder that the MILP naturally captures rather than something specially coded for).

7. Genetic Algorithms — Bowling Rotation & Batting Order

Q49. Genetic Algorithm le system ma ke optimize garcha?

Two separate GAs (`simulate/ga_bowling.py`, `simulate/ga_batting.py`) solve two problems the MILP is structurally the wrong tool for:

- **Bowling rotation** — which bowler bowls each of the 20 overs, maximizing `-mean(runs conceded)` from actually running the ball-by-ball simulator.
- **Batting order** — the permutation of 11 batters (searched within realistic role bands, not a free-for-all), maximizing expected score, or — given a chase `--target` — the fraction of simulated reps that actually reach that target (an approximate chase win-probability).

Both are used because their objectives have **no closed-form formula** (expected runs conceded from a 20-over sequence only exists by actually simulating ball-by-ball dynamics) and their legality is **sequence-dependent** (e.g. "no bowler twice in a row" is a constraint between *adjacent positions*, not a simple count) — a fundamentally different, harder kind of constraint than the MILP's "at least 5 of this type" (`ALGORITHMS.md` §2).

Q50. Bowling rotation GA le kaam garyo tara batting-order GA le kina garena?

This premise doesn't hold against the project's own documentation and test suite, so it's worth correcting rather than answering as asked: **both GAs share the identical core skeleton** (population, truncation selection, elitism, re-verification at higher rep counts before accepting a new "best") and are both covered by an equally rigorous test suite — `tests/test_ga_simulate.py` includes a `test_bowling_rotation_ga_fitness_history_never_regresses` **and** a `test_batting_order_ga_fitness_histor` both passing. Neither `ALGORITHMS.md`, `model-training/README.md`, nor `VALIDATION_REPORT.md` records a finding that the batting-order GA underperforms or fails relative to the bowling rotation GA. What genuinely differs between the two (a real, documented difference — just not a failure) is described in Q52/Q53 below: they encode different kinds of solutions (a sequence of independent choices vs. a permutation) and therefore need different, specifically-matched crossover operators — order crossover for batting, single-point crossover for bowling — which is a design distinction, not one GA "working" and the other not.

Q51. Genetic Algorithm ko convergence kasari evaluate garnu huncha?

Two protections against a genuinely noisy fitness function (`-mean(runs conceded)` over only 8 sampled innings is a small, noisy sample — two equally good rotations can show different scores purely from randomness, the same way 6 heads in 8 coin flips doesn't prove a biased coin):

1. **Elitism** — the single best-known individual always survives unchanged into the next generation regardless of how it happens to score on a fresh random sample, so a genuinely good candidate can never be lost purely to bad luck.
2. **Re-verification** — before accepting a new candidate as the tracked "best," it is re-scored with **3× the reps** (24 instead of 8); the final returned answer is verified with **30+ reps**. This mirrors why a clinical trial needs a large enough sample before declaring a result real.

Convergence is then tracked as the **running best-known fitness**, not each generation's raw top score — deliberately, because plotting the raw noisy top score visibly bounces up and down and can look like "the GA isn't converging" even when the actually-tracked best genuinely never regresses; this was a real, documented bug the project found and fixed by switching to logging the monotonic running-best series instead (`ALGORITHMS.md` §2a).

Q52. Bowling rotation problem ko solution kasari encode garnu huncha?

Chromosome: a flat list of 20 bowler-IDs, one per over (`[bowler_over1, bowler_over2, ..., bowler_over20]`).

Initialization (not random): `_quotas` first splits 20 overs across the pool, giving up-to-5 recognized specialist bowlers their real 4-over quota (matching how a real captain plans overs before a match), then `_sequence` places overs using the classic **"always assign whichever remaining bowler has the most overs left, excluding whoever bowled the previous over"** rule — the standard, provably-correct algorithm for arranging repeated items with no two identical items adjacent (the same underlying idea as the well-known "task scheduling with a cooldown period" problem), guaranteed to succeed whenever the most-frequent item's count doesn't exceed half the total slots.

Crossover: single-point — pick a random over-index k, `child = parentA[:k] + parentB[k:]`.

Mutation: each of the 20 overs independently has a 15% chance of being reassigned to a uniformly random pool bowler.

Repair (Q46): illegal children (over-4-overs bowlers, adjacent repeats) are deterministically fixed rather than penalized or discarded.

Q53. Batting-order GA ko performance limitation kina bhayo?

As established in Q50, no documented evidence exists in this project of the batting-order GA underperforming — so rather than invent a limitation, here is what *is* honestly documented as constraining its search space **by design, not by flaw**:

- It searches **within real historical role bands** (openers / top-middle order / tail) rather than one free permutation of all 11 — a deliberate restriction that shrinks the search space to combinations that make cricket sense (a specialist bowler is never tried at the top of the order), so generations aren't wasted testing nonsensical orders.
- Its fitness signal for a chase scenario (`--target`) is the **fraction of simulated reps that reach the target**, a noisier, more binary signal per simulated innings than the bowling GA's continuous `mean(runs conceded)` — this is a genuinely harder statistic to estimate precisely from a small rep count, which is exactly why the same elitism + 3×-reps-re-verification + 30+-reps-final-check protections (Q51) apply here too, not because the batting GA is weaker in kind.
- A permutation search space is combinatorially large (11! within-band arrangements, even after banding) — inherent to the batting-order problem itself, not a defect of this particular GA implementation.

Q54. Batting-order GA lai improve garna ke garnu parcha?

Not separately itemized in the project's own "What Can Be Done to Make It Better" list (that list's GA-adjacent item is broader: item 5, "a true seed sweep for the production configuration, and a sensitivity analysis over the calibrated scale constants" — `docs/criiq-technical-documentation.md`). Reasonable, codebase-grounded extensions consistent with that list's spirit: (1) applying the same repeated-holdout discipline used for the WPA models — multiple seeds, reported mean ± std — to the batting-order GA's own fitness-convergence claims, closing the "3 runs vs. 15" caveat already flagged for the production WPA models; (2) widening the chase-target fitness signal from a single 0/1 "reached target" outcome per rep to something that also credits *how close* a failed chase got, which would give the GA a smoother gradient to climb, especially at low rep counts; (3) the general item 4 on the project's own improvement list — quantile-regression-style uncertainty estimates — would apply here too, letting a coach see not just the GA's chosen order but how sensitive its win-probability estimate is.

8. Explainability — SHAP and GA Explanations

Q55. SHAP explainability tapai ko system ma kasari work garcha?

SHAP (SHapley Additive exPlanations) provides an **exact, additive decomposition** of one specific WPA prediction: `base_value + Σ(each feature's contribution) = the actual predicted value`, based on the Shapley value from cooperative game theory (build up a prediction feature-by-feature in every possible order, average each feature's marginal effect across all orderings).

The mechanical complication: the shipped `bowling_wpa` model is a `VotingRegressor` blending **two differently-shaped sub-models** — a `RandomForestRegressor` inside a `Pipeline` with a `OneHotEncoder` step, and a `HistGradientBoostingRegressor` that encodes its one categorical feature natively. SHAP's fast, exact **`TreeExplainer`** works directly on the first; the second needs the slower, model-agnostic **Permutation explainer** (perturb each feature against a persisted background sample — `SETTINGS.shap_background_size = 100` training rows, saved as a real training artifact) instead. The two sub-models' SHAP values are then combined using the **exact same `[0.7, 0.3]` averaging weights** the `VotingRegressor` itself uses for its predictions — so the combined SHAP values still sum exactly to the blended model's actual combined prediction, preserving the "exact decomposition, not an estimate" property through a blended model, not just through either half alone (`ALGORITHMS.md` §3.11).

Worked example, in spirit: `base_value = 0.01`, then `career_batting_wpa: +0.04`, `form_bat_sr: +0.02`, `vs_opp_bat_contribution_pct: −0.01`, … summing exactly to the model's actual predicted WPA — a coach learns not just "the model rates her at X" but specifically *what* is driving that number up or down.

Q56. SHAP lai aru explainability methods bhanda kina choose garnu bhayo?

Two reasons, both explicit in the project's own reasoning (`ALGORITHMS.md` §3.11):

1. **Global feature importances are the wrong granularity.** "`career_matches` matters most on average across all players" doesn't answer "why did *this specific player* get *this specific* predicted WPA" — a feature can matter a lot on average and contribute almost nothing for one particular player whose value on that feature happens to sit right at the model's baseline. SHAP is computed *per prediction*, exactly the granularity a coach asking "why is this player rated this way" needs.
2. **The model family already chosen (tree ensembles, Q22/Q23) makes an exact SHAP computation tractable** (`TreeExplainer`), rather than requiring an expensive approximation — a genuinely good match between the explanation technique and the model family already selected for other, independent reasons.

Separately, a plain-language narrative (`recommend/explain.py`) already existed and was **not replaced** by SHAP, because the two answer different questions at different levels of rigor: the narrative is a human-readable *summary* of everything known about a player; SHAP is a mathematically exact *attribution* of one specific number the model produced. `--shap` is opt-in precisely because most callers only need the narrative, and SHAP adds real compute cost (a background-sample permutation pass for the `VotingRegressor` half).

Q57. SHAP explanations correct ra non-misleading chan bhanera kasari validate garnu huncha?

By directly testing SHAP's own defining mathematical guarantee, not by inspection alone. `tests/test_shap_explain.py` includes `test_pipeline_shap_reconstructs_prediction_exactly` and `test_voting_regressor_shap_reconstru` both of which assert `base_value + values.sum() == model.predict(row)` (within floating-point tolerance) — i.e., the test suite mechanically verifies that the additive decomposition SHAP promises actually holds for the exact shipped models, including the harder `VotingRegressor` case where two sub-models' SHAP values are combined with the ensemble's own weights. `test_explain_prediction_output_shape_and_ranking` further checks the output shape and that contributions are meaningfully ranked. This is a stronger validation than "the numbers look plausible" — it's a literal, automated check of the property that makes SHAP

trustworthy in the first place (an approximate or buggy combination of the two sub-models' explanations would fail this test immediately, since the sum would silently drift from the real prediction).

Q58. Selectors ko lagi explainability kina important cha?

Because the system's stated purpose is decision-*support*, not decision-replacement (Q63) — a recommendation a selector cannot interrogate is one they either have to blindly trust or discard entirely, neither of which is useful for a genuinely important, high-stakes call like a national team's XI. `explain.py`'s hard rule ("explanations never state anything that isn't in the computed numbers") and SHAP's exact attribution both exist so a selector can ask "why," get a real, traceable answer, and use their own judgment on top of it — matching the project's own "transparency over black boxes" design principle. It's also the concrete fix for a real, named failure mode the project documents directly: a genuine finisher (RM Ghosh, death-overs strike rate 166) rating low on the composite score reads as a silent contradiction without explanation; with the narrative's explicit transparency note ("this rating is win-probability-added, not raw hitting…") plus the `role_highlights` fallback that still surfaces her as the pool's best finisher, the same fact becomes something a selector can act on rather than dismiss the whole system over.

9. Technology Stack

Q59. Python, FastAPI, React, SQLite, Parquet, and Joblib—jasto technology haru kina choose garnu bhayo?

SQLite (`criiq.db`) — the most consequential and best-documented choice (`docs/criiq-technical-documentation.md`, "Why SQLite"): the entire two-module contract is one file, copyable/diffable/versionable with zero installation; no server process, accounts, or ports; ships in Python's standard library; the real data scale (408 MB, 1.23M rows, 28 tables) fits it trivially; WAL mode gives concurrent reads during a pipeline rewrite; a `mode=ro` URI gives the *engine itself* a hard read-only guarantee for `model-training`, rather than a convention that could be violated by mistake; and real relational features (upserts, unique constraints, foreign keys) that plain CSVs can't provide were directly load-bearing for the identity-resolution fix. PostgreSQL/MySQL were considered and rejected: a server database adds installation, credentials, and networking overhead that nothing in this single-writer, read-heavy workload actually needs.

Parquet — used deliberately *alongside* SQLite, not instead of it, exactly where columnar storage wins: the large, write-once, whole-column-read per-ball datasets (`ball_state_T20.parquet`, `win_prob_T20.parquet`) and the fitted win-probability grids. SQLite is a poor fit for these (no benefit from row-level transactional guarantees on write-once analytical data); Parquet is a poor fit for the rest of the schema (no upserts, no unique constraints, no incremental point updates) — the hybrid uses each format for what it's actually good at.

Joblib — the standard scikit-learn-ecosystem serialization format for persisting fitted models (`model.dump/joblib.dump`); used because the trained artifacts *are* scikit-learn `Pipeline/VotingRegressor` objects, and joblib is the conventional, well-supported way to round-trip exactly those object types (including the `ColumnTransformer/OneHotEncoder` steps inside the pipeline) without a custom serialization format.

FastAPI (backend) — `backend/README.md/model_bridge.py`'s own module docstring gives the reason directly: it keeps a lightweight service (FastAPI + JWT + bcrypt) cleanly separated from the heavy ML dependency stack (scikit-learn, shap, scipy, pandas) that only `model-training` needs — the backend talks to `criiq_models` via a CLI subprocess specifically so the two dependency stacks never have to coexist in one virtual environment, and so a crash inside a Monte Carlo simulation or GA run can't take down the API server (it just returns one failed HTTP request).

React + Vite (frontend) — `frontend/README.md` records the stack (React 18 + TypeScript, Vite 6, React Router 7) as the implemented choice; the specific comparative rationale against alternative frontend frameworks isn't separately argued in the project's own documentation, consistent with this document's practice of only asserting what's actually written down rather than inventing a rationale after the fact — a reasonable, standard choice for a client-rendered SPA talking to a JSON API, but not one the codebase itself defends against named alternatives the way SQLite's choice is.

Why scikit-learn/scipy/numpy/pandas specifically (`docs/criiq-technical-documentation.md`, Appendix A): `scikit-learn` for the model families, pipelines, imputation, and encoding; `scipy.optimize.milp` for exact XI selection and `scipy.stats.spearmanr` for the rank backtest; `pandas` for every group-wise expanding/rolling feature computation; `numpy` for the underlying array math. `xgboost/lightgbm` are present only as **challenger models** in the comparison notebooks (LightGBM also powers the optional `rank-backtest` command), not in the shipped production path.

10. Limitations, Deployment & Future Work

Q60. Tapai ko system ko main limitations ke-ke hun?

Directly from the project's own "Known limitations (honestly reported)" (`model-training/README.md`) and "What is weak or unfinished" (`docs/criiq-technical-documentation.md`):

- **The predictive signal is genuinely small.** Correlation ≈0.07–0.12, R^2 near zero, MAE that barely (sometimes doesn't) beat a mean-predictor baseline. Per-match WPA is dominated by variance the features cannot see (conditions, luck, opposition-specific plans).
- **Style and keeper coverage is partial.** Bowling style resolved for ~33% of real female T20 bowlers and ~43% of male; wicket-keeper flags cover 35 women's (18 nations) + 36 men's (12 nations) players — not the full player pool either way, though it degrades gracefully (no crash, no guess) rather than failing.
- **Some pipeline stages are easy to forget in automation.** `wpa` and `role-finalize` are not part of the main `refresh` orchestrator; a naive scheduled job running only `refresh` would silently serve stale WPA labels/roles.
- **Several composite-score scaling constants are calibrated by inspection, not fitted or formally sensitivity-tested** — documented, which mitigates but doesn't remove the arbitrariness.
- **Win-probability v1 is coarse** — step-shaped bins, no team-strength or match-conditions prior. Since WPA (the training label itself) derives from this model, its quality caps every downstream WPA-based metric.
- **The production models' own variance estimate rests on fewer seeds (3) than the challenger comparison (15)** — a real, flagged asymmetry in how thoroughly the *shipped* configuration's stability was itself re-checked.
- **Men's domestic T20 leagues remain switched off** (`enabled=False` in `config.py`) — men's recommendations currently run on international data only, with no domestic-form signal, unlike women's.
- **No live, in-progress match ingestion** — the win-probability model and full simulator both work, but nothing currently scores a match *as it happens*; `simulate` runs a hypothetical match end-to-end, and the Matches feature only shows already- fully-recorded results.
- **Two admin backend routes have no auth guard** (`GET/POST /auth/admin/pending-coaches`) — a real, confirmed gap in the web layer, unrelated to the modeling pipeline, worth knowing before any production exposure.

Q61. Future work ma tapai le ke-ke improve garna plan garnu bhayeko cha?

The project's own ordered list (`docs/criiq-technical-documentation.md`, "What Can Be Done to Make It Better"), roughly by expected value:

1. **Improve the label before the model** — replace the empirical win-probability grid with a calibrated ML model (team strength, venue norms, era effects); WPA quality is capped by this model, so this would likely raise every downstream metric's ceiling at once.
2. **Automate the out-of-band stages** (`wpa`, `role-finalize`) into the main `refresh` orchestrator.
3. **Richer context features** already derivable from existing data: venue scoring environment, opposition strength at match time, tournament stage, days-since-last- match.
4. **Uncertainty estimates** (quantile regression / prediction intervals) — letting the engine distinguish "confidently mediocre" from "high-variance wildcard."
5. **A true, matched seed sweep for the production configuration** plus a formal sensitivity analysis over the calibrated scale constants.
6. **Close remaining coverage gaps** — continued style enrichment, broader wicket-keeper coverage, a legitimate full ICC rankings source.
7. **Enable men's domestic T20 leagues** (IPL/BBL/PSL/CPL) — scaffolded, not yet ingested.
8. **Revisit learning-to-rank properly**, with role-specific relevance labels instead of one collapsed grade — its earlier loss to regression (Q20) may not hold under a fairer labeling scheme.

9. **Operational hardening** — CI running both test suites, scheduled refreshes with freshness alerts, database schema versioning.
10. **Serve it properly** — the webapp exists; the render/explain layer was built to be consumed by something friendlier than a terminal (largely realized since this list was written — see `WORKING.md`).

Also directly actionable and separately flagged: a real-match win-probability graph (the ingredients — ball-by-ball history and the persisted win-probability model — already exist; nothing currently replays a real match through them), and closing the two unguarded admin routes.

Q62. Actual cricket board ma yo system kasari deploy garnu huncha?

As it stands, the project ships a **self-contained demo deployment**, not a hardened production one — worth stating precisely rather than overclaiming. `docs/DOCKER-HANDOVER.md` documents the actual mechanism: day-to-day development uses a root `docker-compose.yml` that bind-mounts `criiq.db` (452 MB) and `model-training/artifacts/` (32 MB) as untracked, pipeline-owned data; for handing the system to someone else, a separate **bundle image** (`backend/Dockerfile.bundle`) layers those two directories directly into the backend image so the recipient can run one command with no separate data setup — explicitly *not* standard production practice (baking a 450 MB database into an image conflates data and code lifecycles), but the right trade for a demo handover. Images are architecture-specific (built separately for `linux/amd64` vs. `linux/arm64`).

For a real board deployment, the honest gap list (already implied by §"Limitations" and the project's own "What Can Be Done" item 9, "Operational hardening") would be: a scheduled, monitored `refresh` (not a manual re-run), the two unguarded admin routes fixed, a database refresh cadence that doesn't silently skip the `wpa/` `role-finalize` stages, and — if the data volume or concurrent-user count grew materially beyond the current single-writer/read-heavy pattern — a re-evaluation of the SQLite choice against that changed workload, which the project's own documentation already frames as the trigger that would make a different choice correct.

11. Reflective Questions (for you to answer, grounded in real facts)

These two questions are personal to your own experience building this project — an honest viva answer here has to come from you, not from the codebase. What follows is not a script; it's the concrete, verifiable material from this project you can build a genuine answer from, so whatever you say stays traceable to something real rather than a generic claim.

Q64. Project ma tapai ko personal contribution ke thiyo?

Talking points genuinely grounded in this codebase's own commit-level history and documentation (all attributable to work recorded in the repository under your authorship — `docs/criiq-technical-documentation.md` itself lists you as author):

- Diagnosing and fixing the **cross-gender identity collision** (32 Cricsheet identifiers shared between real male and female players) at the schema level (`UNIQUE(cricsheet_identifier, gender)`), and auditing every derived-table builder to key on `(identifier, gender)` — removing 2,842 corrupted rows on rebuild.
- The **contribution-share → WPA** pivot, triggered by a concrete wrong ranking, and the full WPA regression pipeline (features, leakage prevention, model selection) built around it.
- The **repeated-holdout (3 fractions × 5 seeds) evaluation discipline** applied consistently to every model/hyperparameter decision, including the honest rejection of experiments that didn't survive it (age feature, bowling containment features, unregularized forests).
- The **149-match genuine forward-holdout validation** (`VALIDATION_REPORT.md`), including catching and correcting your own earlier mistaken conclusion in §9 — a real example of self-correction under evidence, worth mentioning directly since the report itself documents it.
- Extending a women's-first system to a **fully verified, equally-tested men's T20I scope**, finding and fixing two real bugs in the process (unscoped model artifacts, hardcoded gendered pronouns in narrative text).
- Building the **transparent composite-score-plus-MILP** recommendation engine and its explainability layer (SHAP integration across a blended `VotingRegressor`, which needed real engineering, not just calling one library function).

Q65. Tapai le project bata sabai bhanda important learning ke gain garnu bhayo?

Concrete, checkable candidates worth reflecting on honestly (pick what actually resonates from your own experience building this, rather than all of them):

- **A theoretically appealing choice isn't automatically the right one until it's checked on real data** — three separate instances in this project alone: WPA beat contribution-share (a real, measured failure motivated the switch, not intuition alone); regression beat a purpose-built learning-to-rank model on the actual selection task; `max_features="sqrt"` helped one gender scope and was confirmed, via the same rigor, to *not* generalize to the other — each decision was made from a real, repeated measurement, not from which approach sounded more sophisticated.
- **Correctness bugs hide in real, messy data, not in algorithms** — the cross-gender identity collision, the recall-flag bug that kept a player retired since 2012 in a 2026 recommendation, the gender-blended selection-rate denominator that mislabeled genuine Nepal men's regulars as fringe players — every one of these was found by inspecting real output, not by a unit test on synthetic data alone.
- **Honest evaluation is a discipline, not a formality** — always benchmarking against a naive baseline (mean-predictor, random-XI, constant win-probability), reporting a weak number as weak (R^2 near zero, stated plainly rather than buried), and catching your own earlier wrong conclusion (`VALIDATION_REPORT.md` §9) are all the same underlying habit, and arguably the single most transferable skill from a project like this one.

Appendix — Source Map

Every answer above traces to one or more of these files; consult them directly for more depth or to re-verify a number before quoting it in your defense:

Topic	Primary source
Overview, principles, strengths/weaknesses, future work, dependencies, glossary	`docs/criiq-technical-documentation.md`
Algorithms, formulas, worked examples (MILP, GA, WPA, SHAP, shrinkage, Monte Carlo)	`model-training/ALGORITHMS.md`
Engineering log — composite score, feature list, model tuning history, bugs found/fixed	`model-training/README.md`
File-by-file function reference (model-training)	`model-training/CODE_WALKTHROUGH.md`
Data sources, identity resolution, idempotency, registry logic, bugs found/fixed	`data-collection/README.md`
File-by-file function reference (data-collection)	`data-collection/CODE_WALKTHROUGH.md`
Cross-stack request tracing (frontend → backend → model-training)	`WORKING.md`
Real-data validation results, including the 149-match forward holdout	`model-training/VALIDATION_REPORT.md`
Docker/demo deployment mechanics	`docs/DOCKER-HANDOVER.md`
MILP/selection correctness tests	`model-training/tests/test_selection.py`
GA correctness/convergence tests	`model-training/tests/test_ga_simulate.py`
SHAP exactness tests	`model-training/tests/test_shap_explain.py`